

PGR

Globální viditelnost

prof. Ing. **Adam Herout**, PhD.

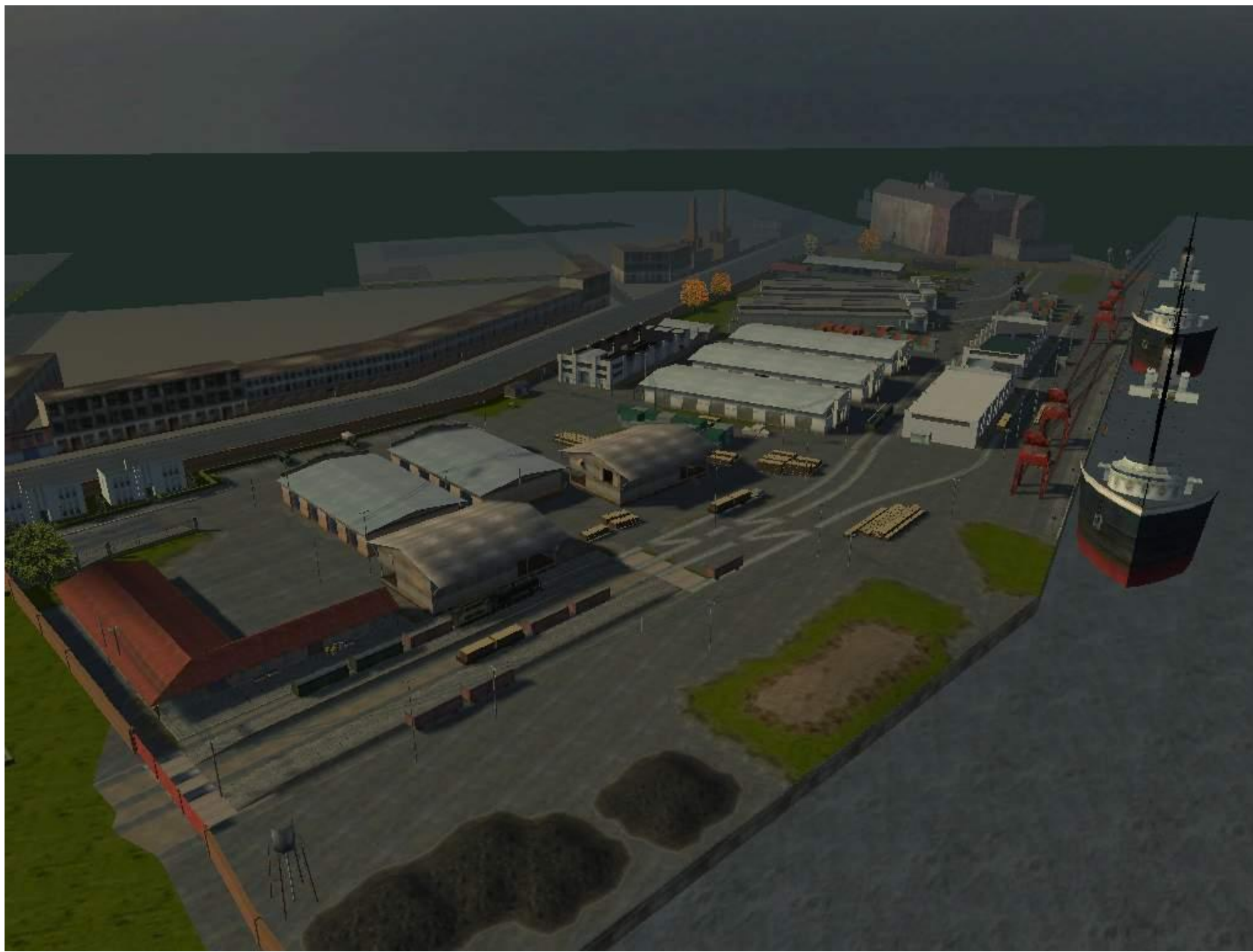
- Urychlení
 - Kreslení pouze viditelných těles
 - Překreslení pouze nutných částí
 - Úrovně detailu
- Pořadí vykreslování
 - Když není Z-buffer
 - Průhledné objekty
 - Jiné optické jevy
- Efekty
 - Promítání do textury
 - Stíny
 - Motion blur

Překreslení pouze nutných částí

- Nepoužívá se 3D
- Pouze 2D regiony
- S použitím HW ustupuje do pozadí

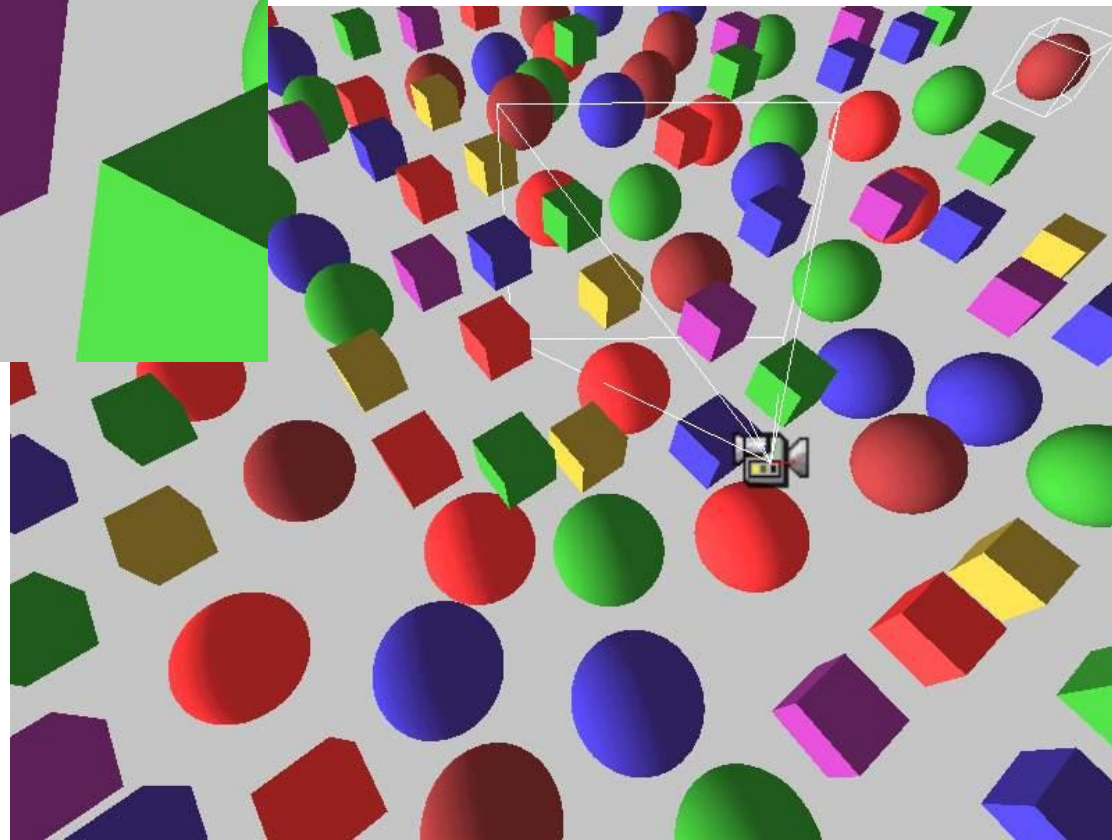
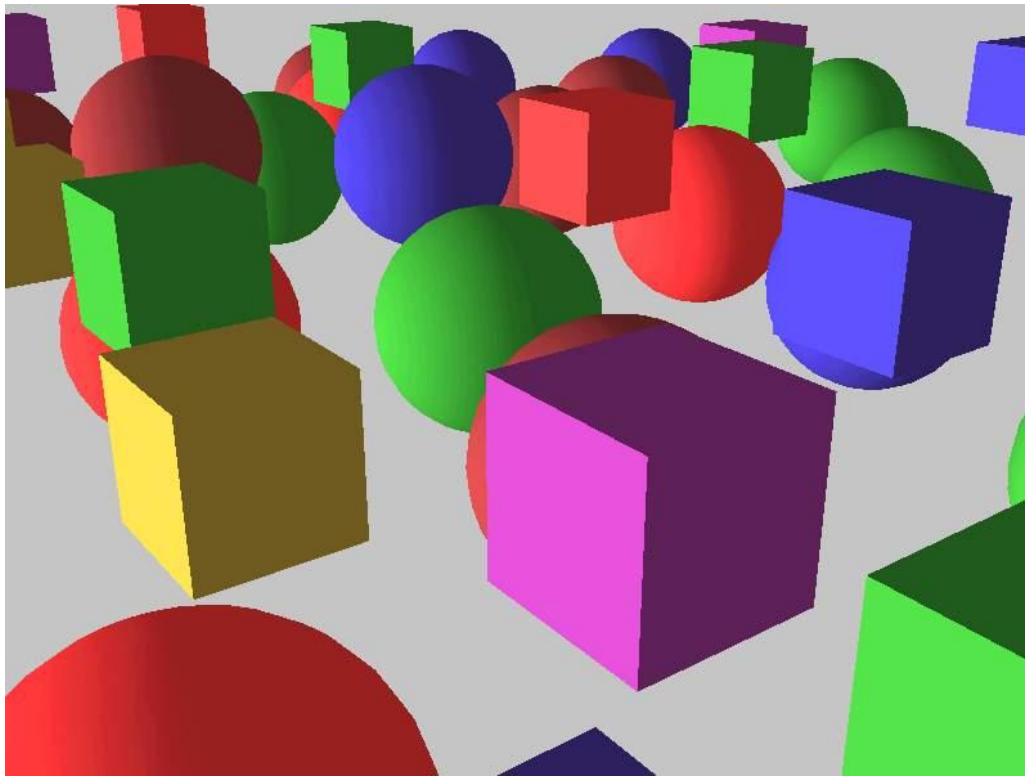
Kreslení pouze viditelných těles

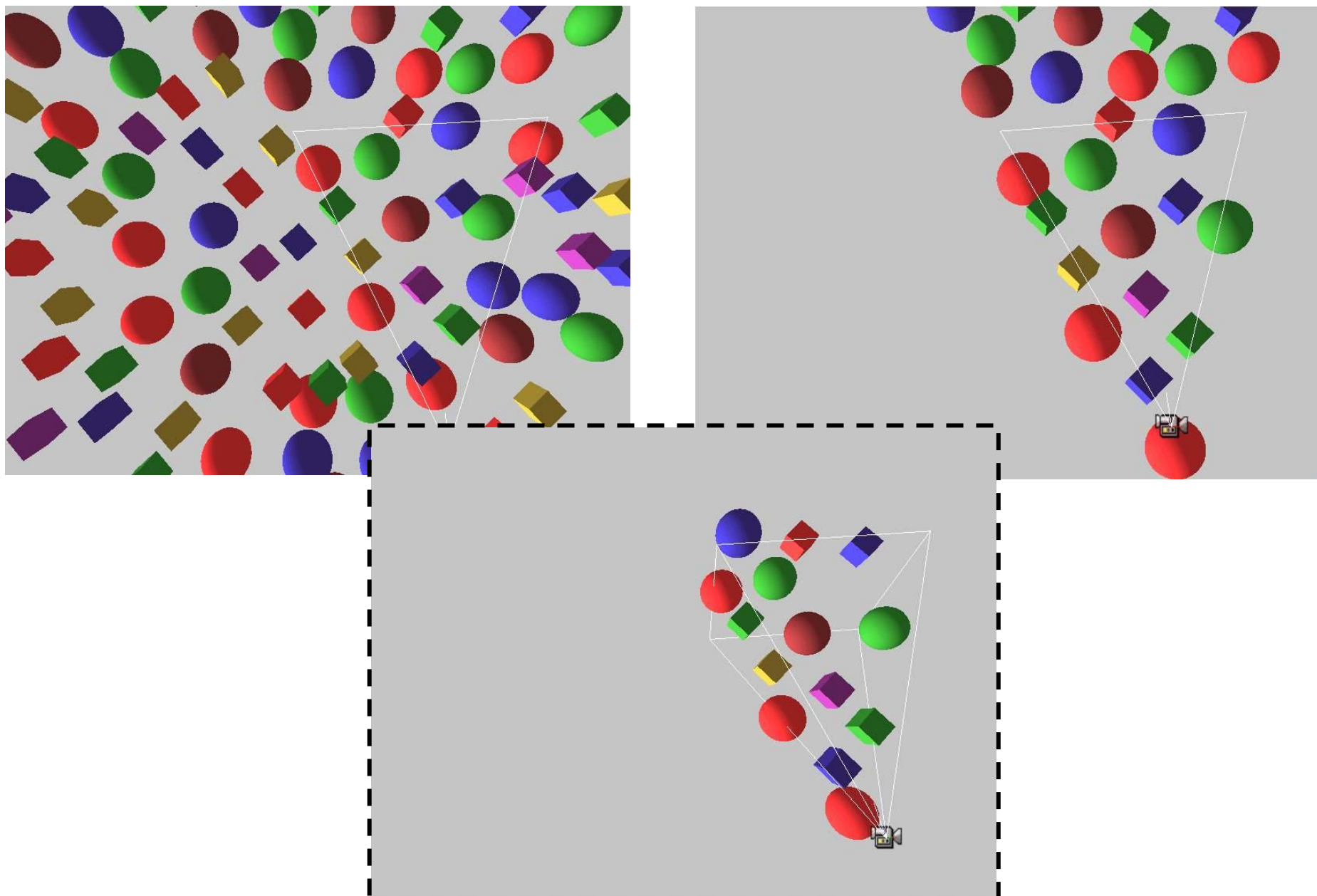
- Nekreslit tělesa mimo zorné pole
 - Podle stran
 - Podle near a far
- Nekreslit zakrytá tělesa
- Hierarchické dělení scény

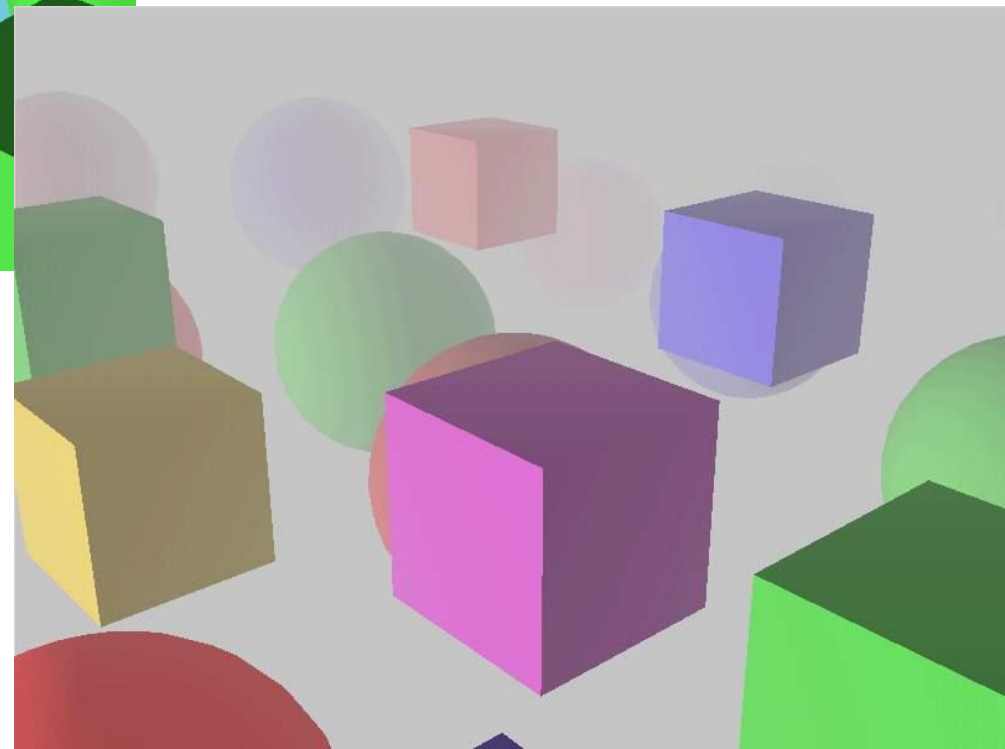
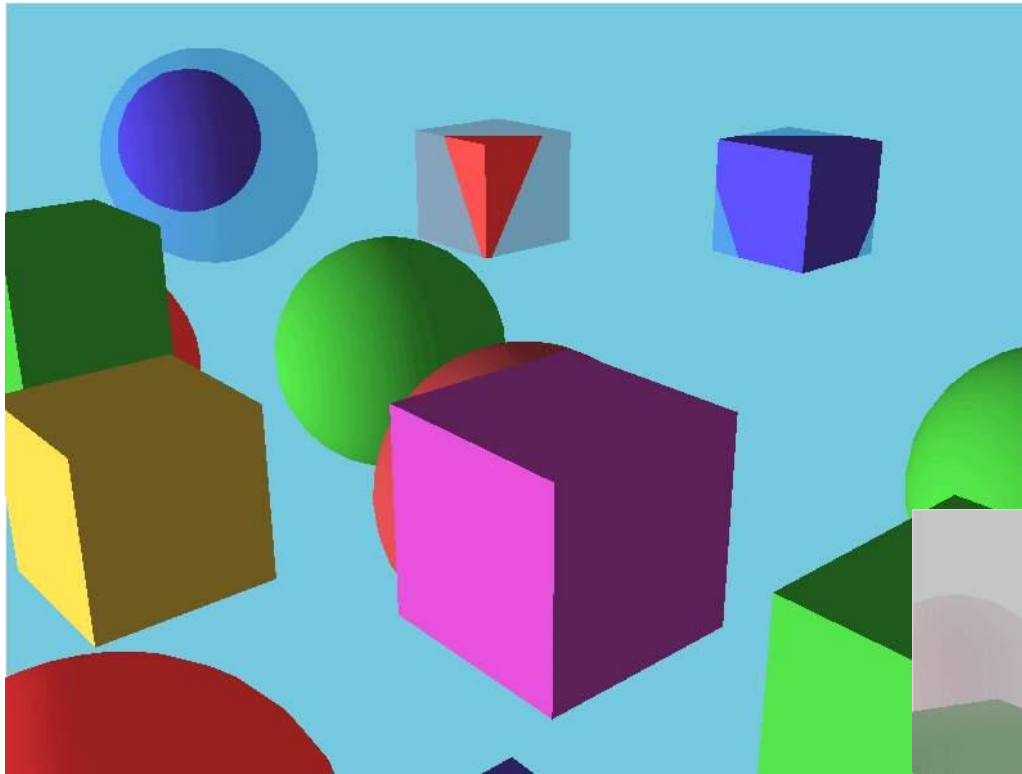




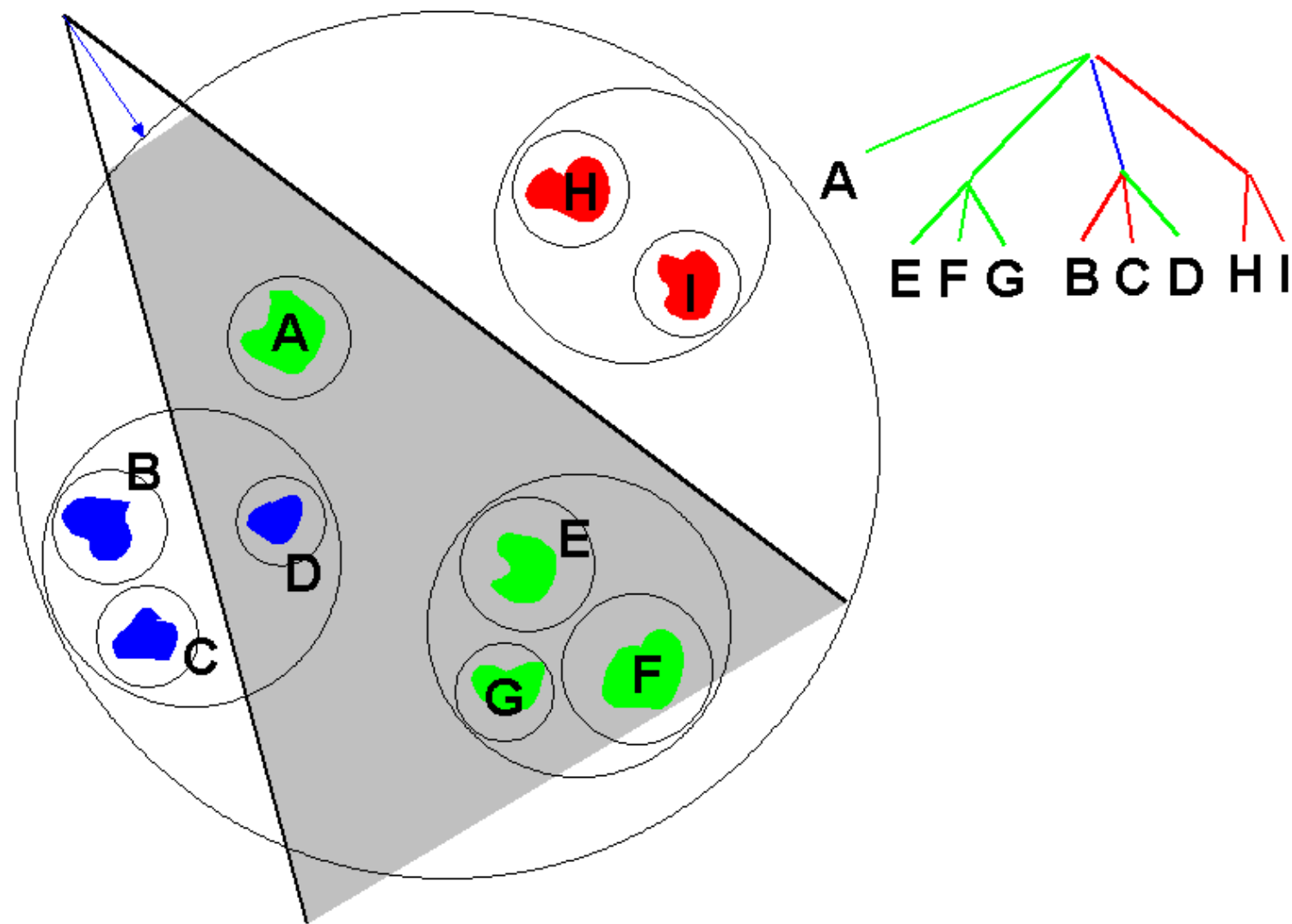




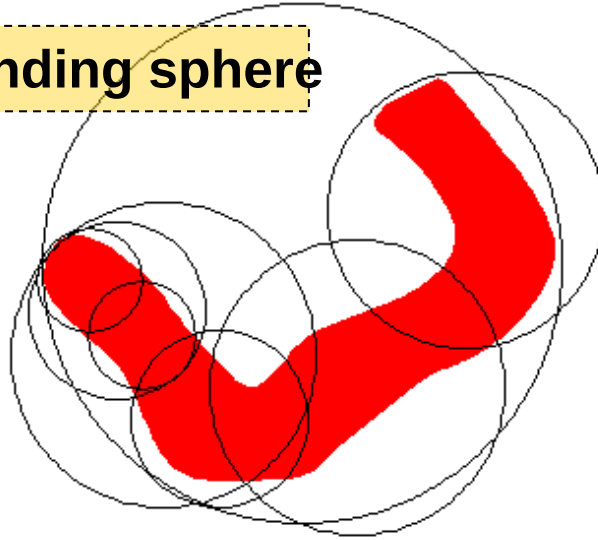




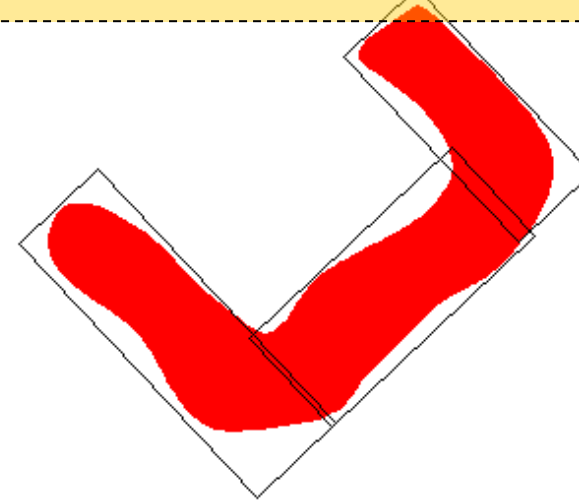
- Hierarchie:
 - Žádná
 - Stromy
 - Mřížky
 - Kombinace



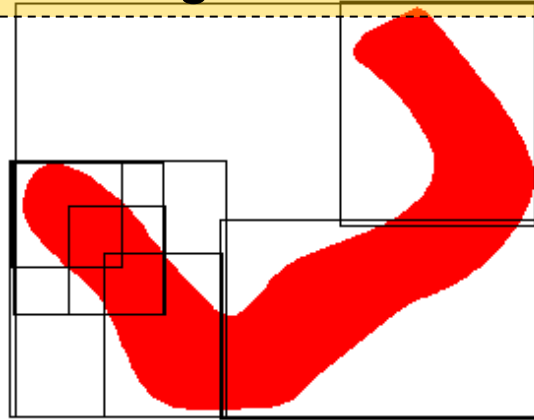
Bounding sphere



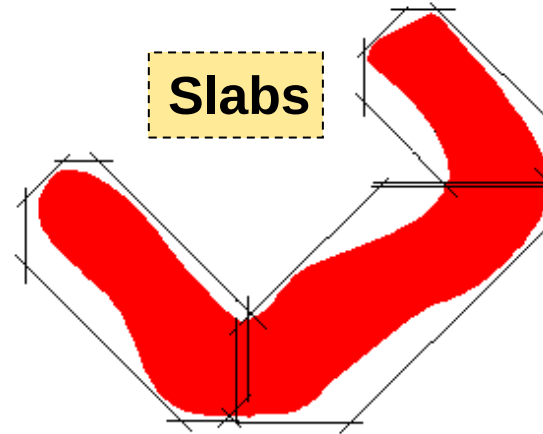
OBB – Oriented Bounding Box

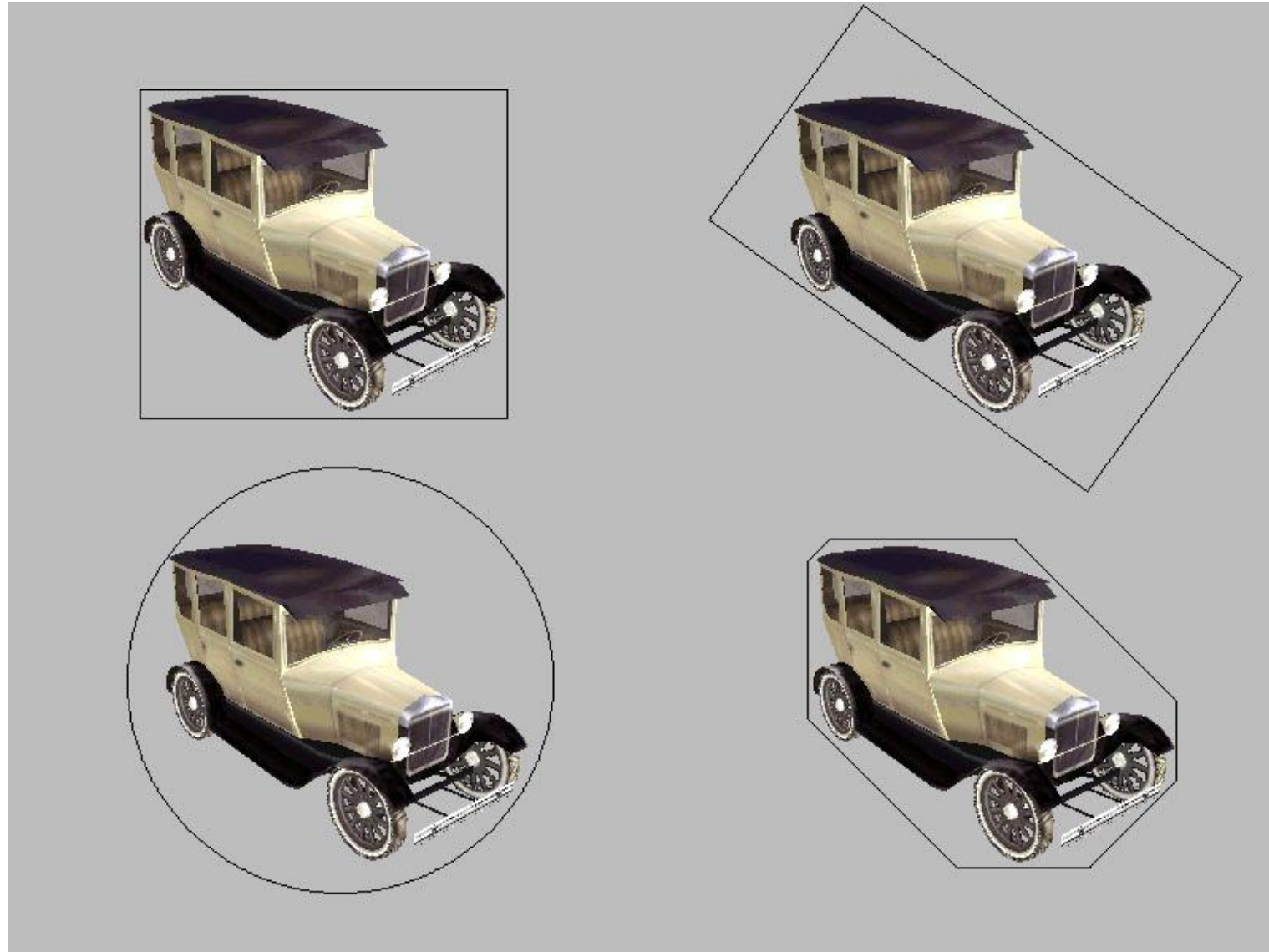


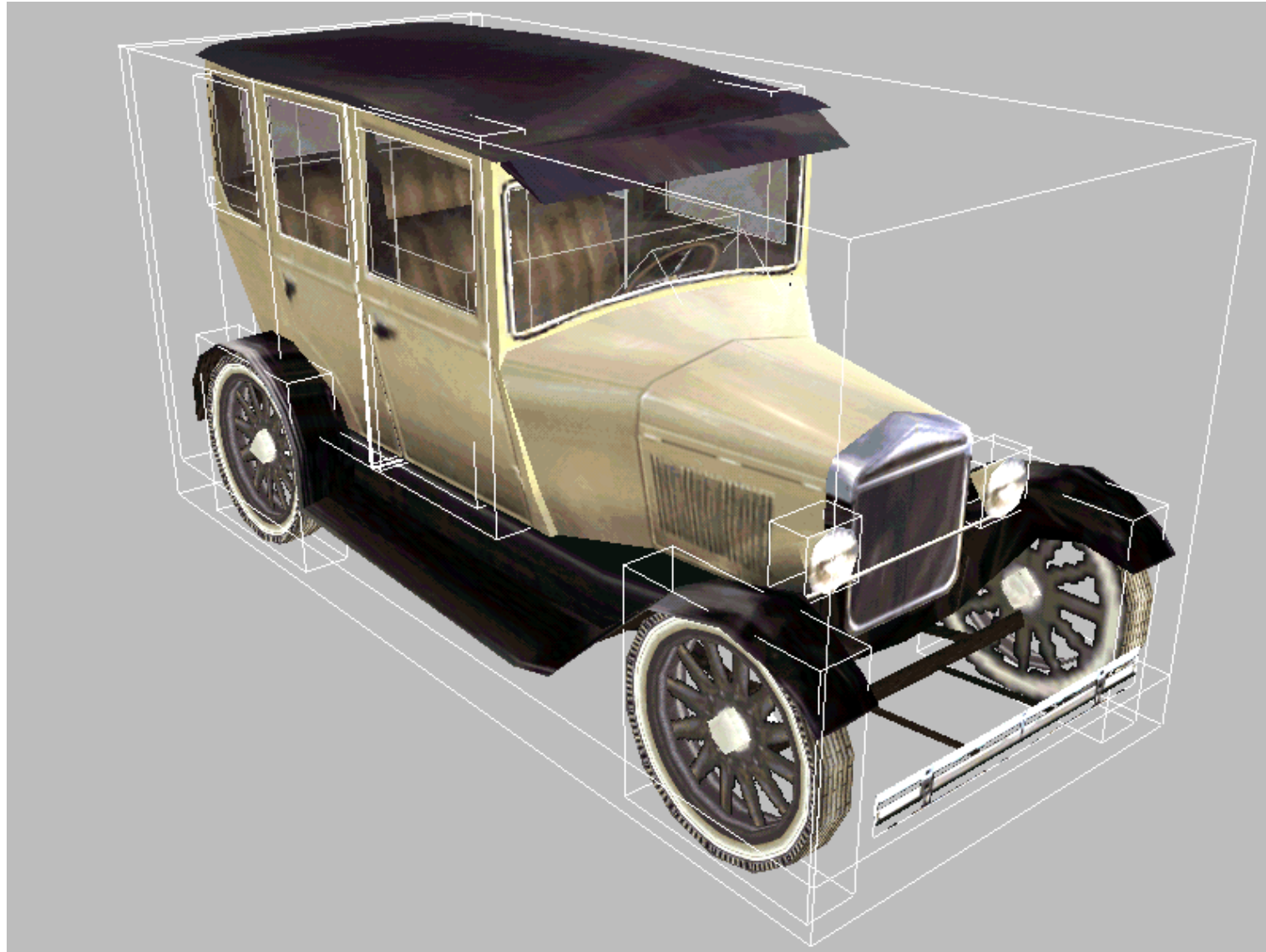
AABB – Axis-Aligned Bounding Box



Slabs

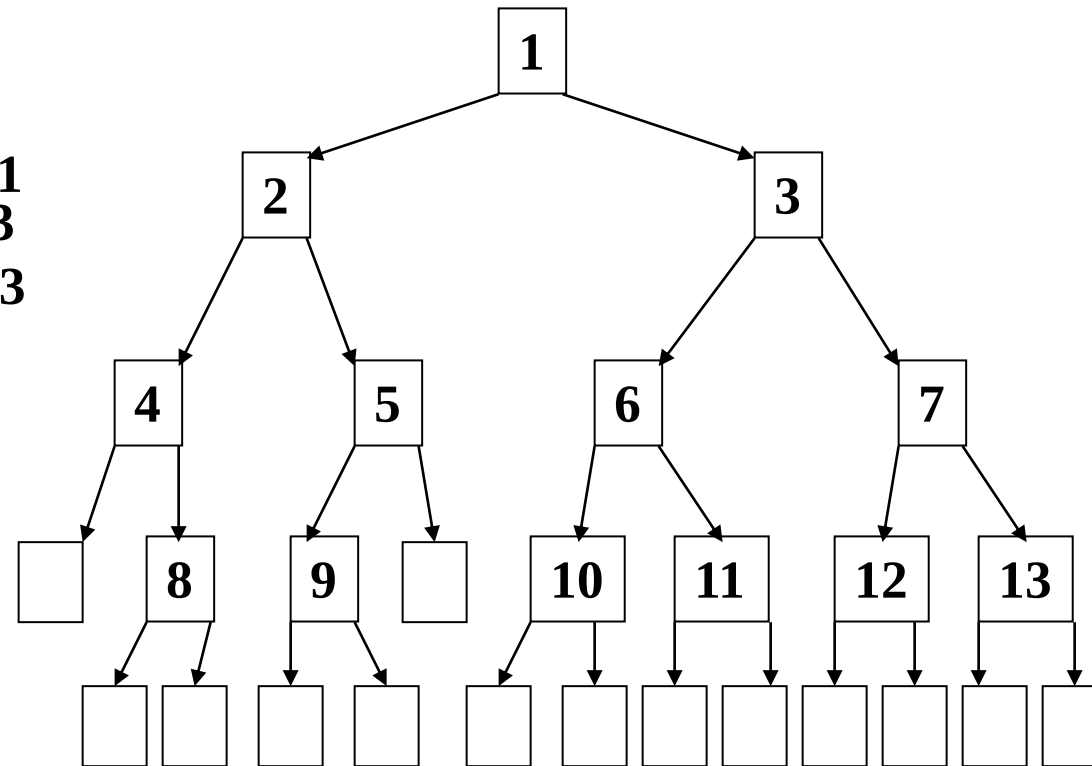
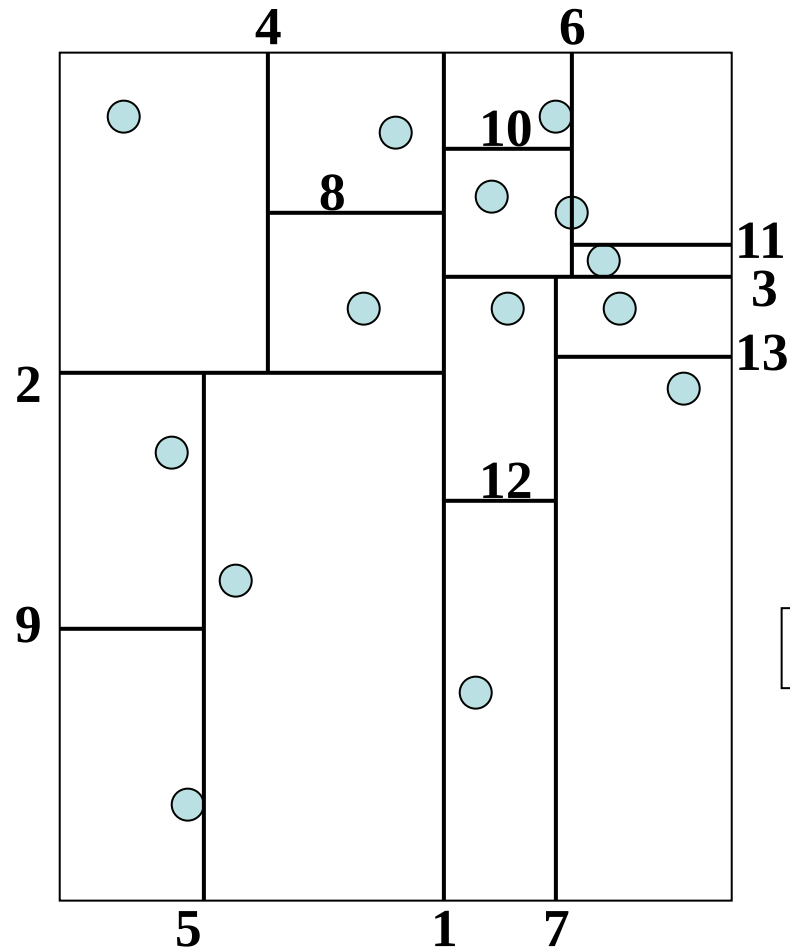






- BSP stromy
 - Axis-aligned × Polygon-aligned
 - Nejsou dostatečně flexibilní pro zobrazení dynamických scén; výpočet BSP stromu může trvat příliš dlouho

- <http://www.rolemaker.dk/nonRoleMaker/uni/algogem/kdtree.htm>

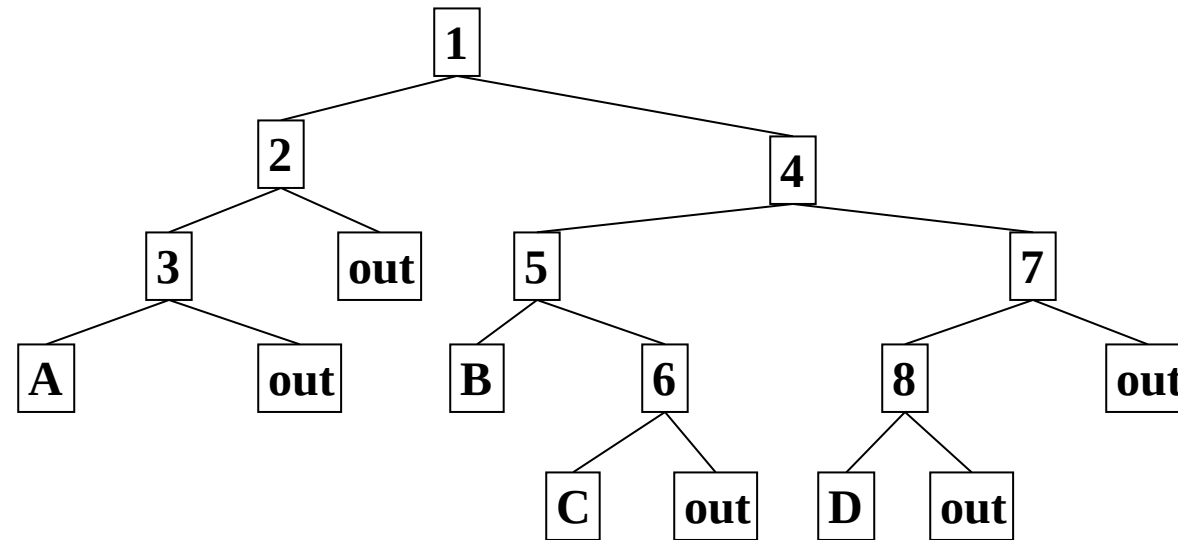
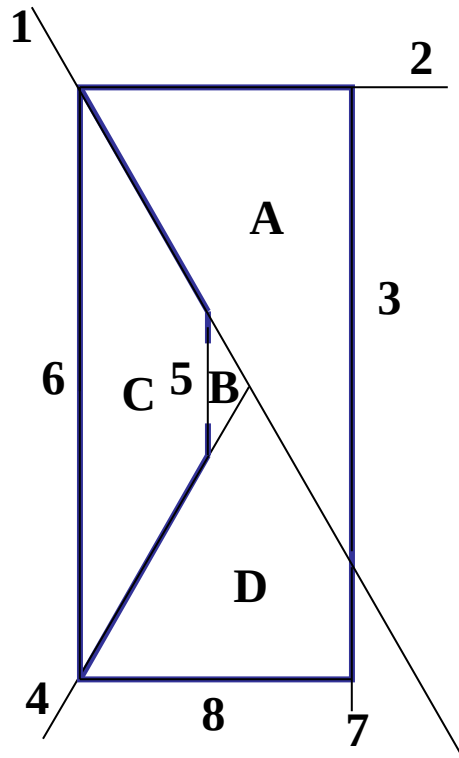


PostavStrom (buňka B , hloubka H)

1. Pokud je hloubka H stromu příliš velká nebo buňka B již obsahuje malý počet těles, skonči.
2. Rozděl buňku B na menší části B_i a učiň z nich následníky uzlu (buňky) B .
3. Pro každou z buněk B_i dělej:
 - (a) Do buňky B_i zařaď odkazy na ta tělesa, která do ní alespoň částečně zasahují.
 - (b) PostavStrom (B_i , $H + 1$).
4. Zruš odkazy na všechna tělesa v buňce B .

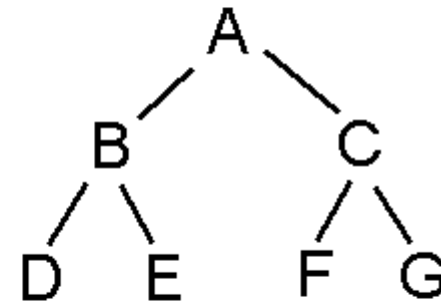
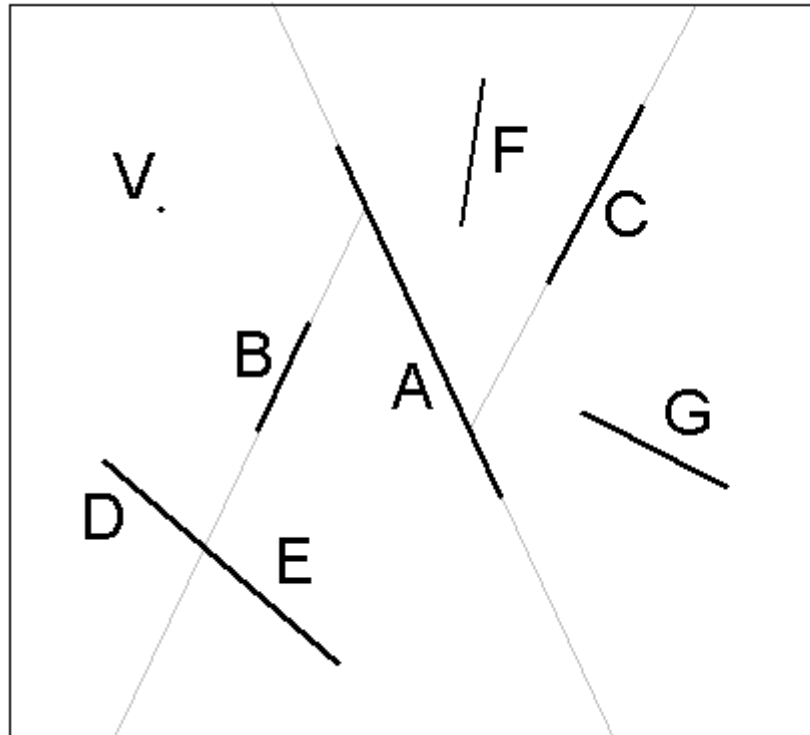
Algoritmus 14.1: Rekurzivní stavba stromové struktury pro uspořádání scény

- BSP tree
 - Každý uzel je rozdělením prostoru rovinou

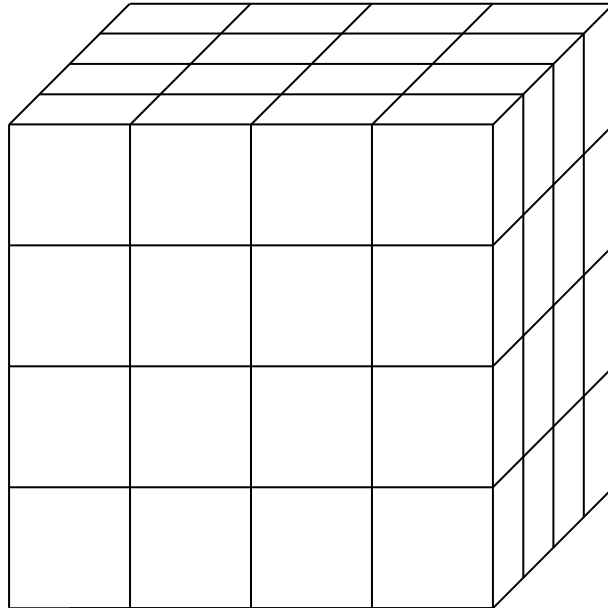


- <http://symbolcraft.com/graphics/bsp/index.html>

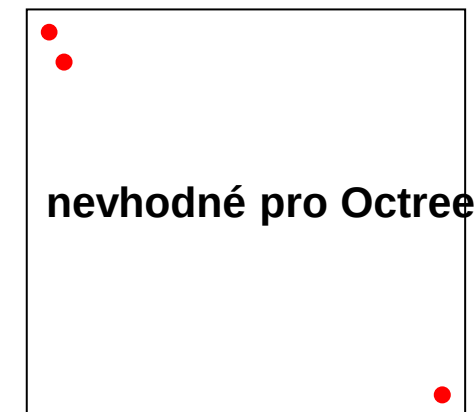
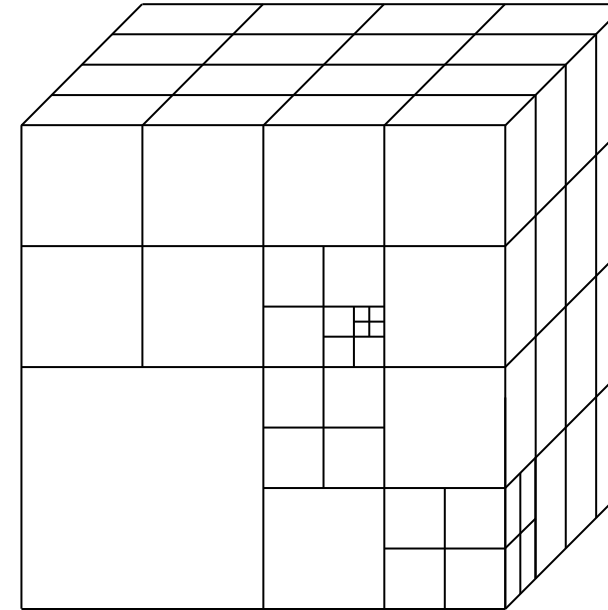
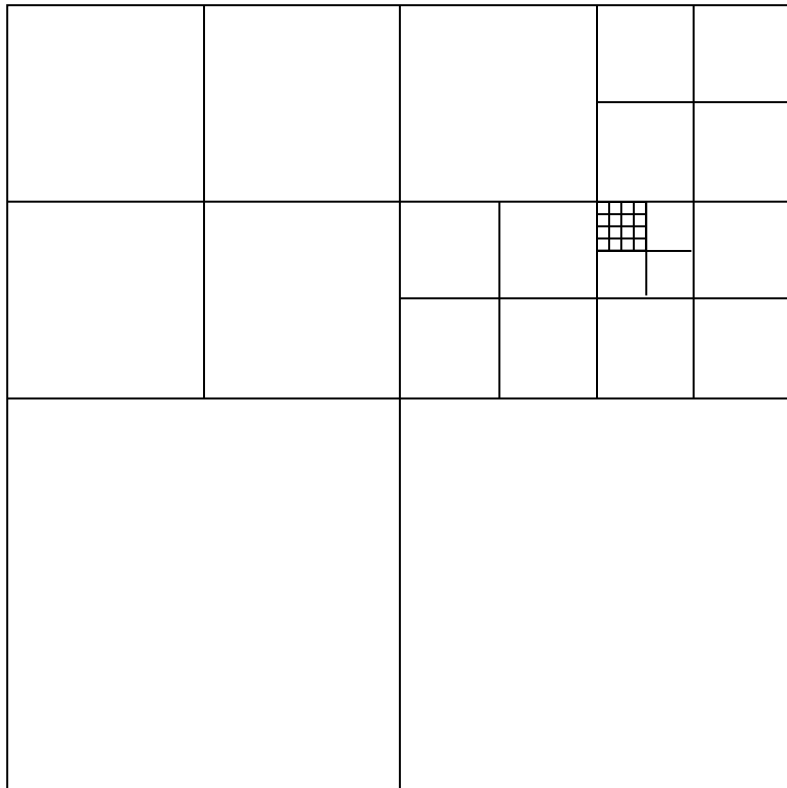
- Polygon Aligned BSP trees
- Užitečné vlastnosti:
 - Pro daný pohled V jej můžeme procházet odzadu dopředu, lze tedy použít tzv malířův algoritmus, bez použití Z-Bufferu



- Efektivní průchod
- Problém: „čajník uprostřed fotbalového stadiónu“

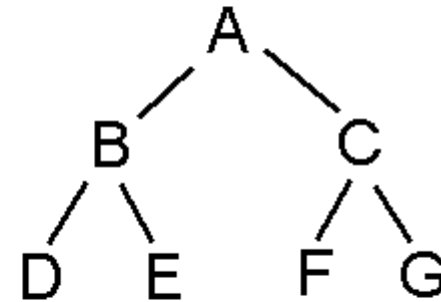
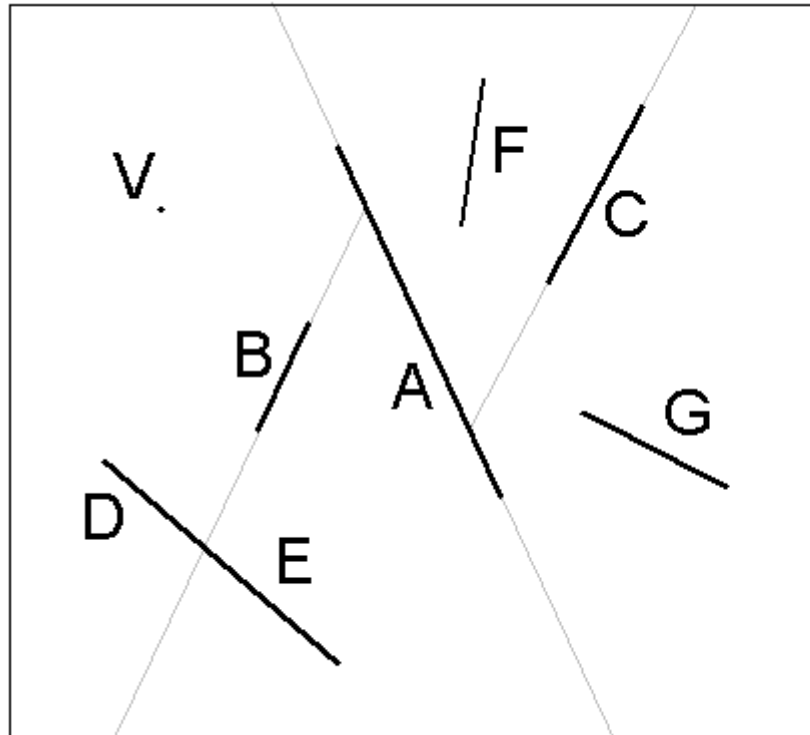


- Quadtree → Octree
- Relativně snadný průchod
- Adaptivní

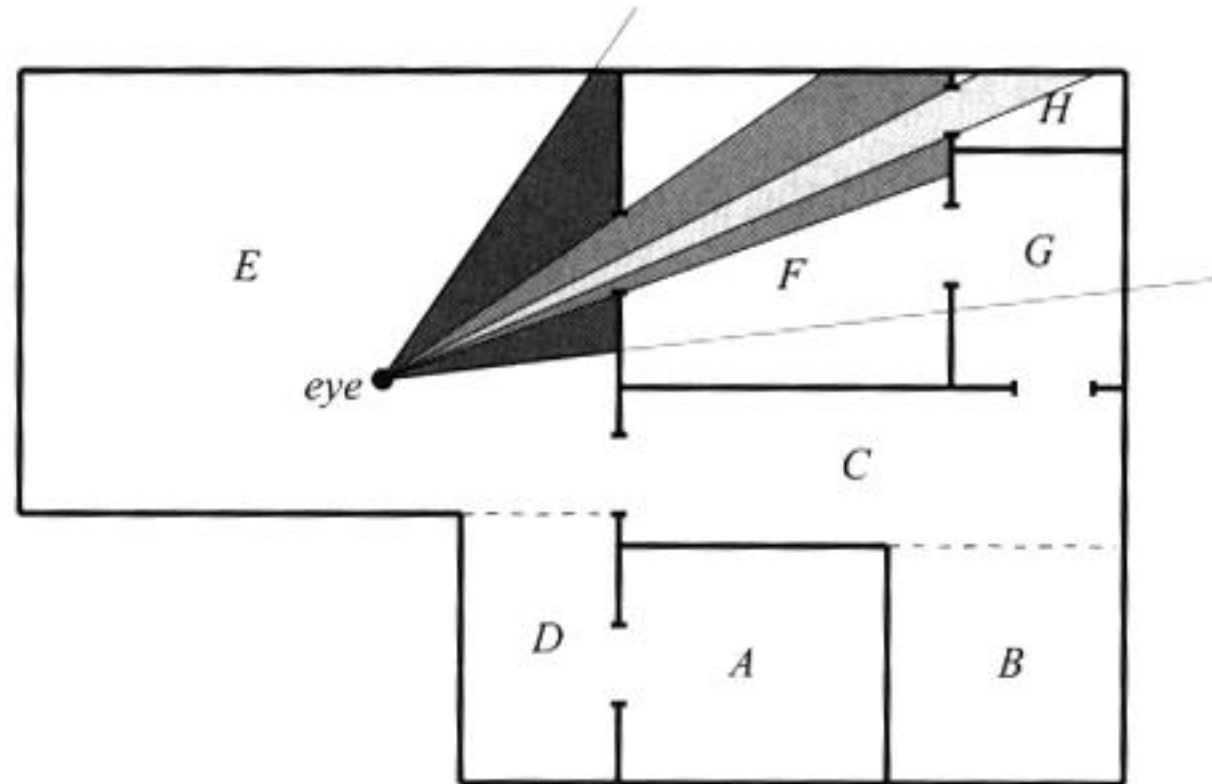


- Jak vypočítat předem a „uložit“?
- Tělesa, která nemůžeme vidět (z dané části prostoru)
- Tělesa skrytá za jinými

- Pro daný pohled V jej můžeme procházet odzadu dopředu, lze tedy použít tzv malířův algoritmus, bez použití Z-Bufferu
- V každém uzlu lze uložit informaci o viditelnosti jednotlivých objektů



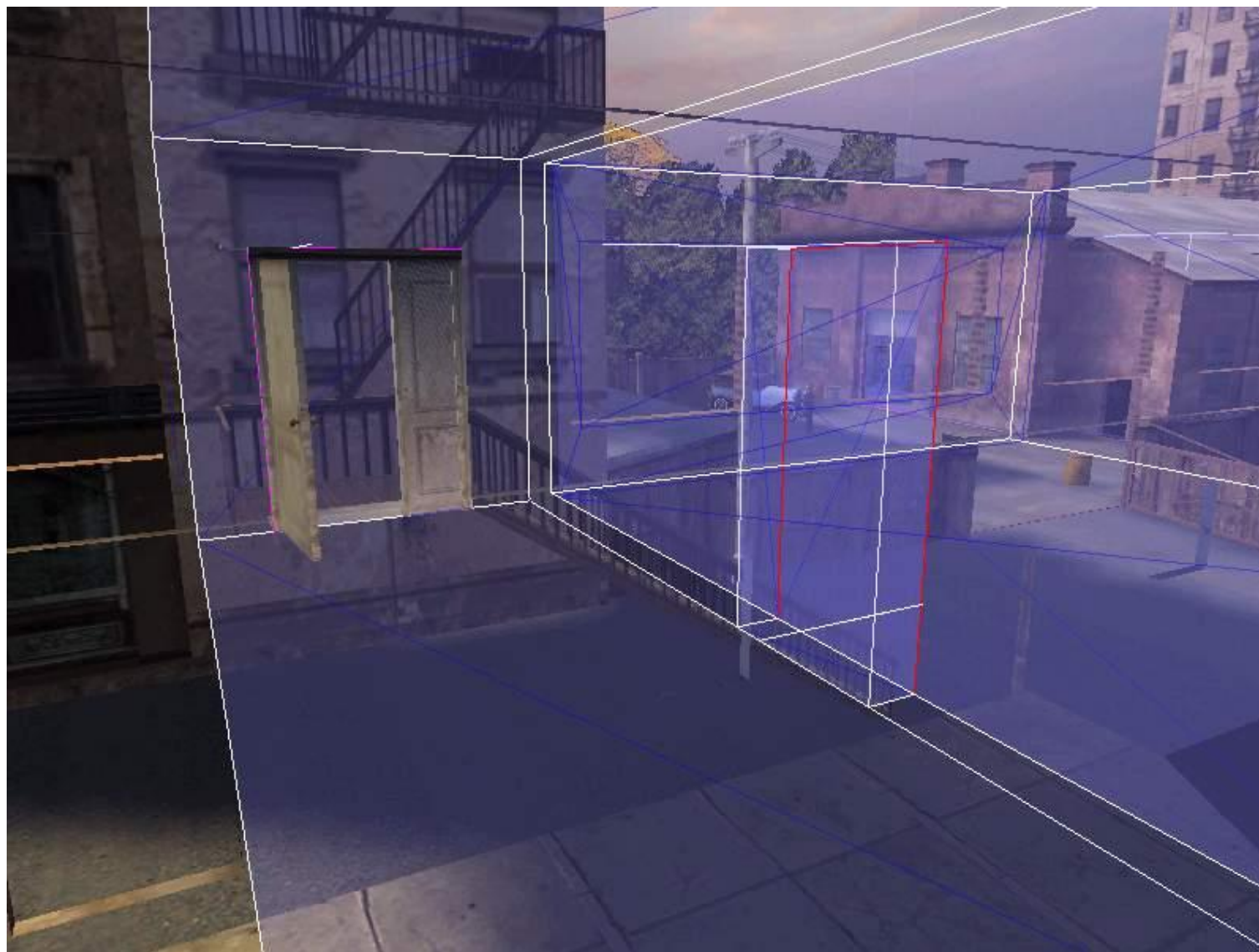
- Příklad rozdělení scény na buňky spojené portály:
 - Pozorovatel je v buňce E
 - Sousedící buňky: C, D, F
 - Z pohledového objemu a pozice portálů vidíme, že sousedící buňky D a C nejsou viditelné a nejsou dále zpracovávány



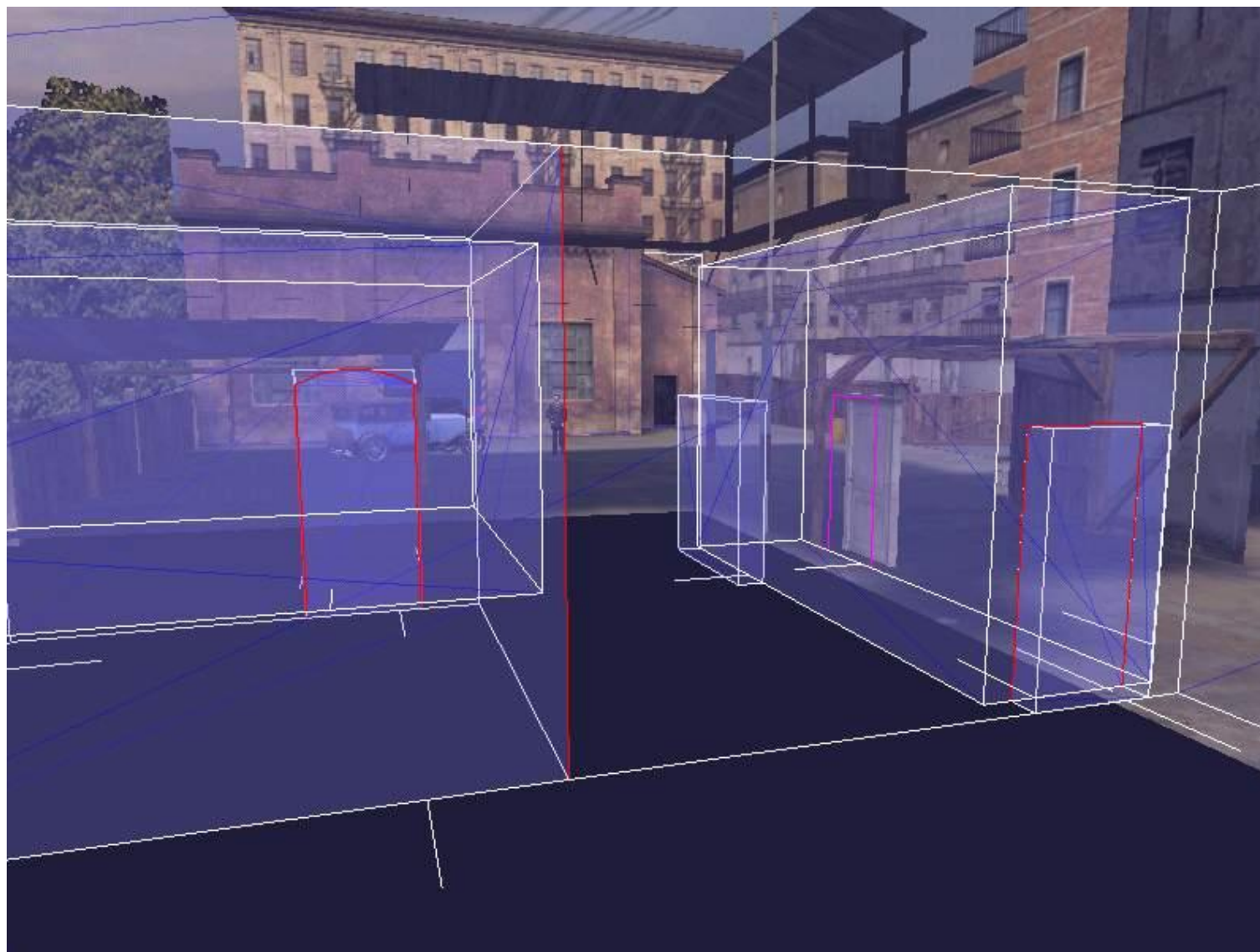
- Výhodné pro zobrazování městské zástavby
- Scénu rozdělíme na buňky (cells) – obvykle místnosti – okna, či dveře – portály – spojují jednotlivé místnosti
- Tuto informaci použijeme pro vytvoření tzv. množiny potenciálně viditelných objektů – Potential Visible Set – PVS
- do jedné struktury uložíme:
 - Stěny buňky
 - všechny objekty uvnitř buňky
 - informace o sousedních buňkách a
 - informace o portálech, které je spojují
- Existuje řada metod pro zobrazení takovéto struktury

- Algoritmus *Luebke a Georges* (1995) pro zobrazení scény s buňkami a portály:
 1. Najdi buňku V ve které se nachází pozorovatel
 2. Inicializuj dvojrozměrný bounding box (BB) P = obdélník obrazovky
 3. Zobraz objekty buňky V , se zapnutým ořezáním podle pohledového objemu.
Pohledový objem je daný pozorovatelem a P
 4. Rekurzivně pokračuj pro sousední buňky:
 - Pro každý portál najdi 2D projekci a její AABB
 - Spočítej průsečík P a AABB
 5. Pro každý průsečík:
 - Je-li prázdný pak buňka spojená tímto portálem není vidět; tedy dál nepokračuj
 - Existuje-li neprázdný průsečík, pak zobraz obsah sousedící buňky ořezaný pohledovým objemem, který je daný pozorovatelem a zmíněným průsečíkem
 6. Pokud byl průsečík neprázdný, pak buňky, které sousedí se sousední buňkou mohou být též viditelné: rekurzivně pokračuj pro v kroku 3

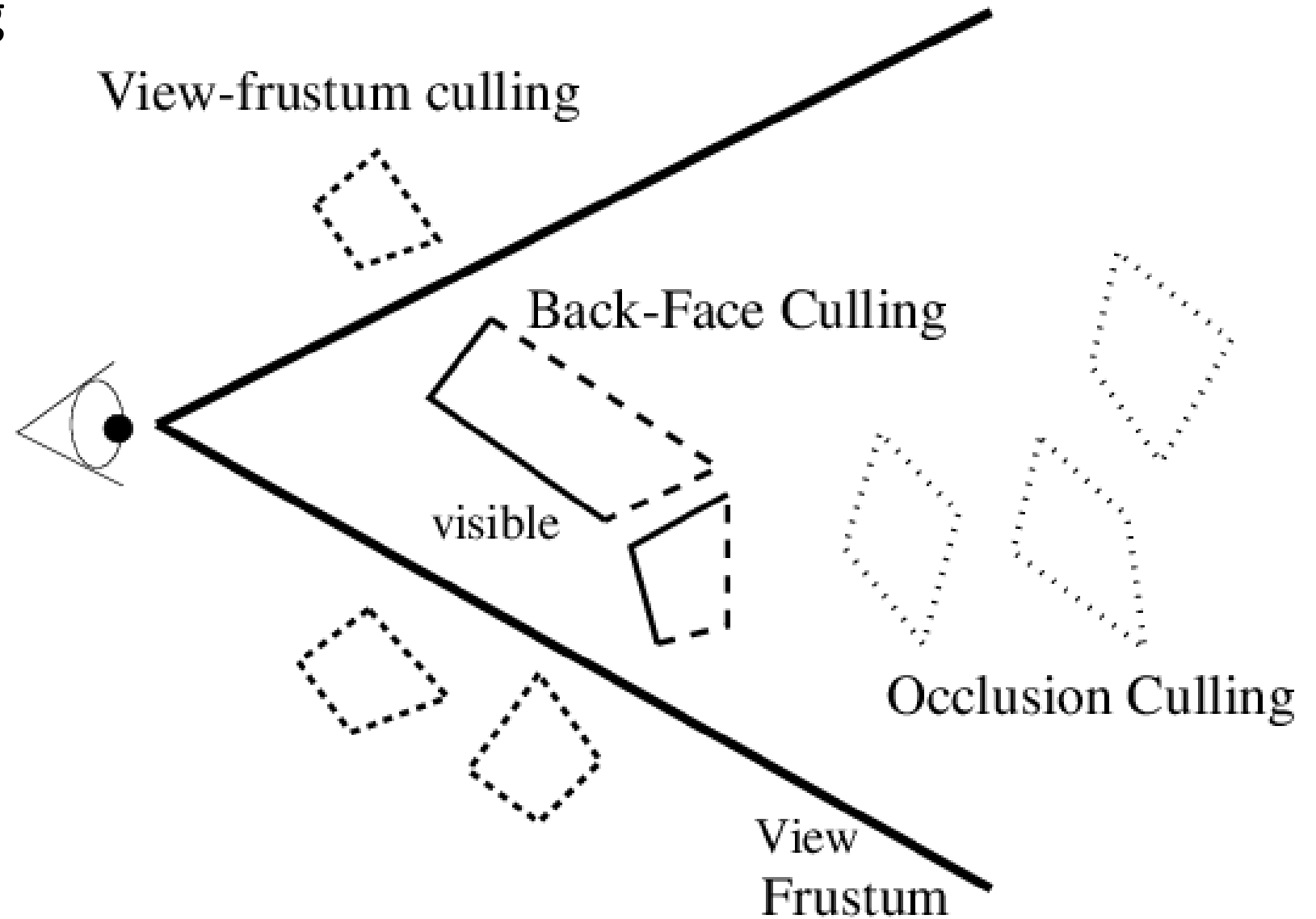




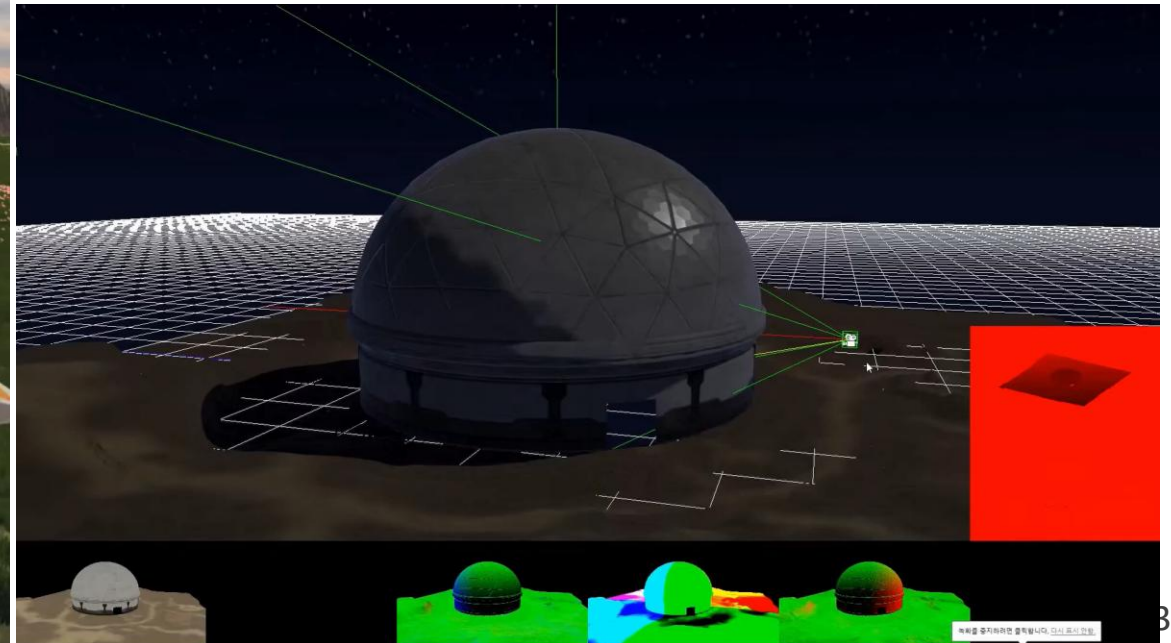




- View-Frustum culling
- Back-Face culling
- **Occlusion culling**



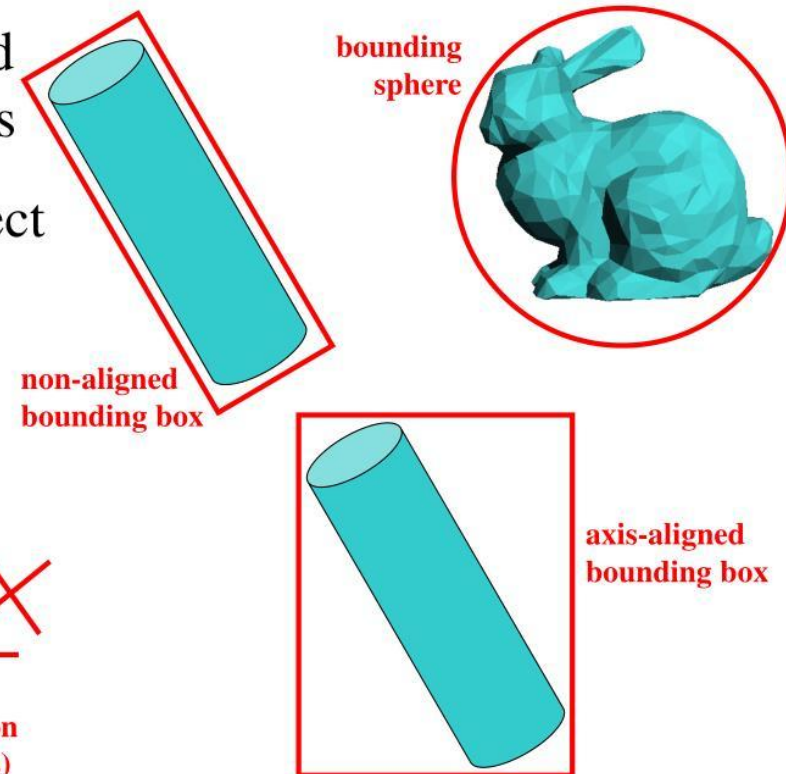
- **Goal:** Remove hidden objects
- Precomputed vs. **runtime**
- **Hardware occlusion Queries & Hi-Z**



- Proxy render + Query
- Post-depth sample count
- Async decisions & Conditional render

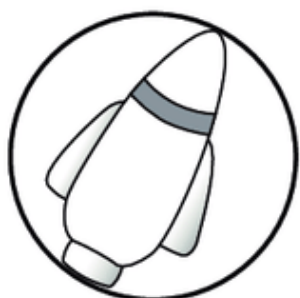
Conservative Bounding Regions

- tight → avoid false positives
- fast to intersect

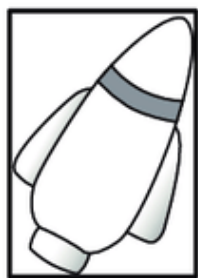


BETTER BOUND, BETTER CULLING

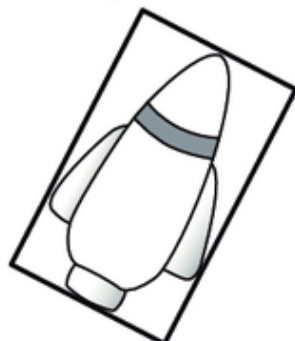
FASTER TEST, LESS MEMORY



SPHERE



AABB



OBB



8-DOP



CONVEX HULL

- Depth pyramid (max-depth 2x2 mipmap)
- Built after depth pre-pass
- Compute-driven culling
- Early-Z hardware rejection

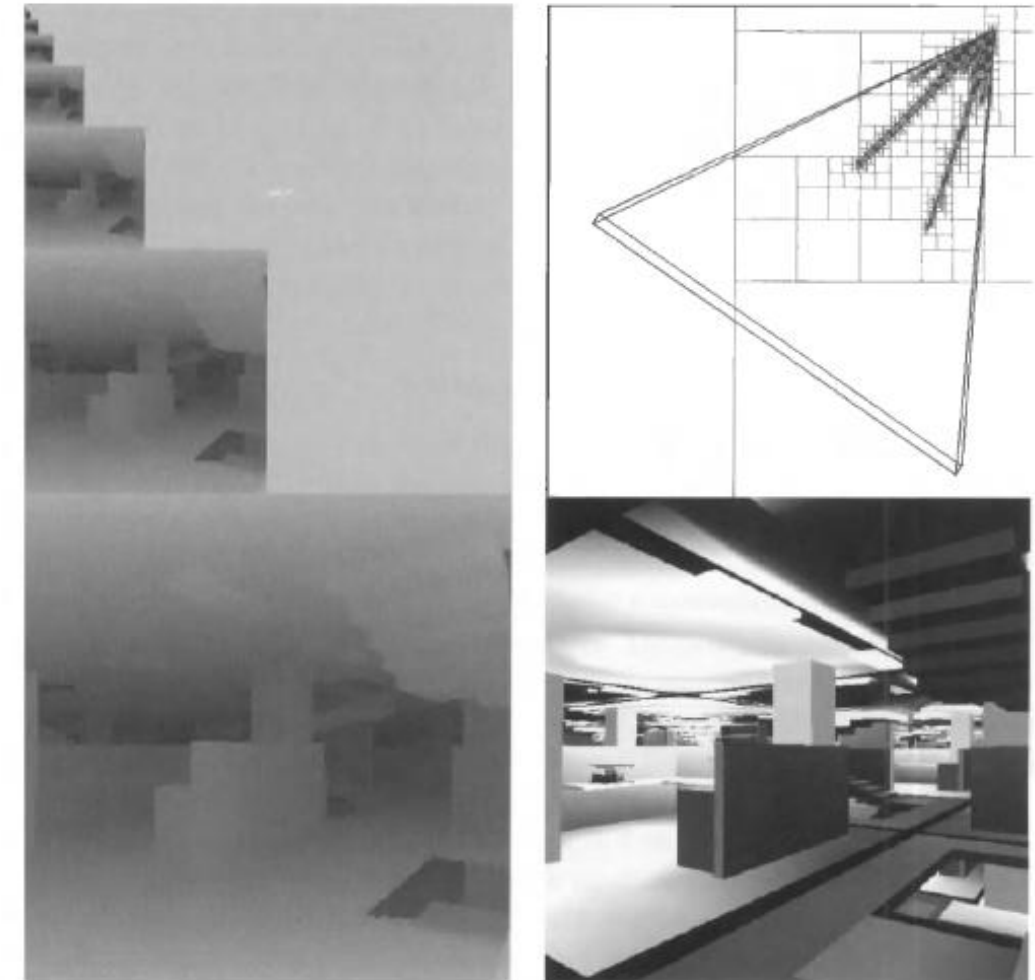
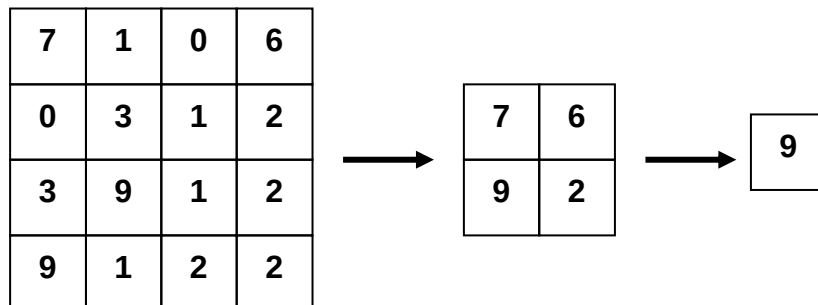


Figure 14.19. Example of occlusion culling with the HZB algorithm [450, 452], showing a complex scene (lower right) with the corresponding Z-pyramid (on the left), and

Hierarchický Z-Buffer

```
ProcessOctreeNode(OctTreeNode N)
if(isOccluded(NBV, ZP)) then return;
for each primitive p in N
    tileInto(p, ZP)
end
for each child node C in N do front-back
    ProcessOctreeNode(C)
end
```

tileInto ... zobraz polygony v listovém uzlu do scény a Z-pyramidy

isOccluded ... přední strany BB se testují vůči Z-pyramidě. Uzel je zakrytý, pokud všechny přední strany BB jsou zakryté.
Začneme na nejhrubší úrovni, ve které je průmět stěny menší než buňka Z-pyramidy

Octree

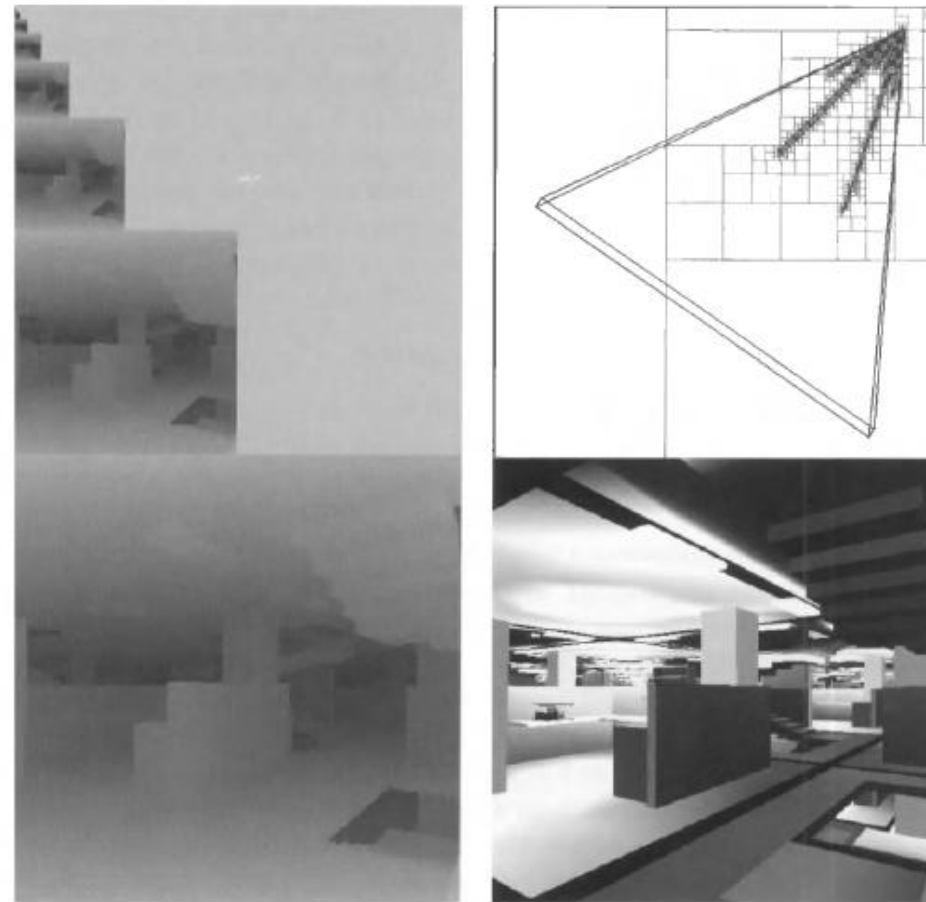
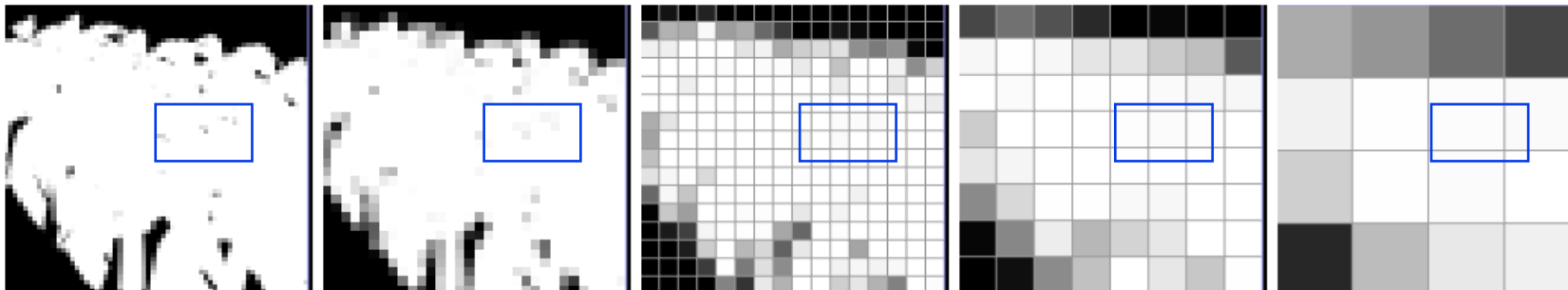
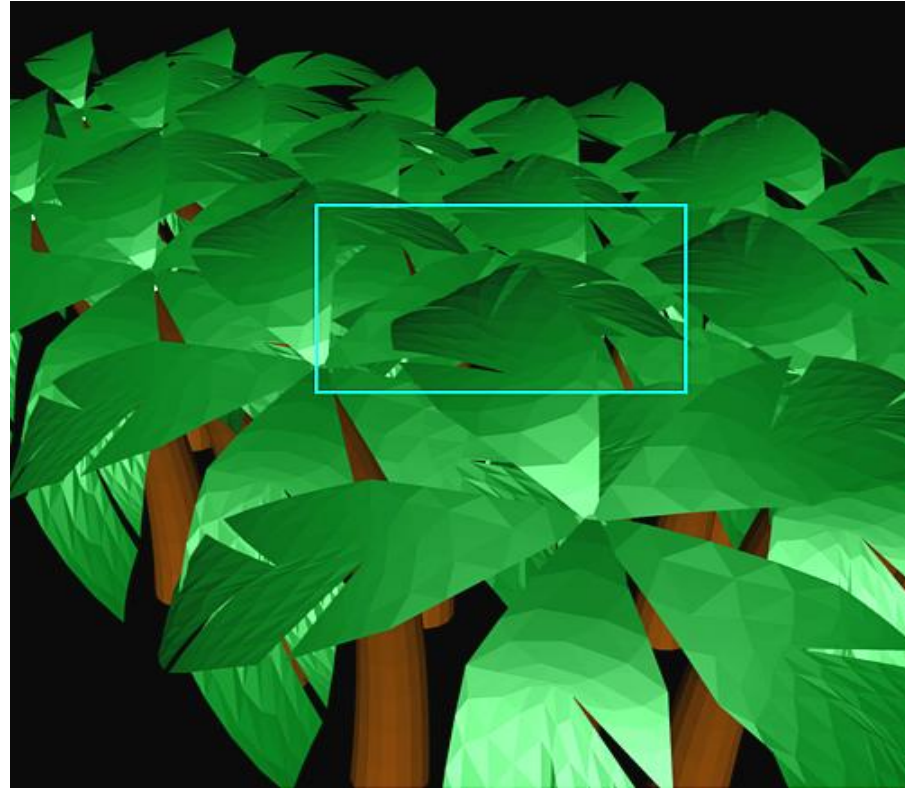


Figure 14.19. Example of occlusion culling with the HZB algorithm [450, 452], showing a complex scene (lower right) with the corresponding Z-pyramid (on the left), and

Aspect	CPU Occlusion Culling	Hardware Occlusion Queries (HOQ)	Hierarchical Z (GPU)
Execution Platform	Runs entirely on the CPU using software bounding-volume tests. No GPU queries are needed.	Runs on the GPU , issuing occlusion queries on simplified geometry (e.g. bounding boxes) for each object.	Runs on the GPU using a depth-buffer hierarchy. The scene depth is downsampled into a mipmapped “Hi-Z” pyramid which shaders use to test object bounds.
Overhead / Performance	Significant CPU workload: needs to maintain spatial data (octrees, BSP, etc.) and test many objects. Relies on spare CPU cores (e.g. Umbra middleware) but can become a CPU bottleneck.	Extra GPU work: each query requires additional draw calls (often a simple proxy geometry) bracketed by query commands. This adds CPU–GPU synchronization and stalls if not carefully managed.	GPU-side cost of building and sampling the Hi-Z pyramid each frame. This uses many parallel GPU threads, so it scales well for large numbers of objects, but the depth-pyramid construction is a nontrivial GPU overhead.
Latency	Immediate (same-frame) . Since tests run on the CPU, occlusion decisions are available within the same frame.	Delayed (multi-frame) . Query results often arrive 1+ frames later (e.g. Unreal Engine reads back visibility one frame later). Naive usage causes CPU–GPU stalls or “popping” artifacts due to this asynchrony.	Synchronous (single-frame) . Tests are done using the current frame’s depth buffer (via the Hi-Z map), so visibility can be resolved within the same frame without multi-frame delay.
Accuracy / Conservativeness	Exact (non-conservative) . Since the CPU knows the exact geometry, it can determine visibility precisely (aside from coarse bounding tests).	Conservative . Uses simple proxies (e.g. bounding boxes), so it can overestimate visibility (“object is probably visible”). It never falsely culls a visible object, but it may fail to cull some actually hidden geometry.	Conservative . Uses coarse depth blocks from the mipmap, so it may under-cull (fewer objects get culled). In practice this means that more objects are rendered instead of the visible objects being incorrectly discarded.
Typical Use Case	Static or moderately dynamic scenes where CPU has idle capacity. Widely used in middleware (Umbra) for large scenes. Good when GPU time is precious.	Dynamic scenes with complex occluders (city environments, fully dynamic geometry). Works online and handles deformable occluders. Common default in engines (e.g. Unreal) for movable objects.	GPU-driven rendering pipelines with massive object counts. Suitable for culling thousands of objects in parallel on the GPU. Often used in GPU-based engines or when batching many small objects (e.g. instanced foliage).



- Ořezání tělesem záběru
- Binární dělení prostoru (BSP)
- Uniformní mřížka, Octree
- Portály
- Occlusion culling – hierarchický depth-buffer